

GESTÃO DE TCC COMO SISTEMA DISTRIBUÍDO

Coreografia de eventos sobre ZeroMQ brokerless, com apoio de IA

O Cenário 3 pede uma solução para a gestão de TCC, hoje fragmentada e burocrática.

OPERACIONAL

Acompanhar o progresso; controlar versões de documentos (proposta, parciais, final); compor bancas avaliadoras; agendar defesas.

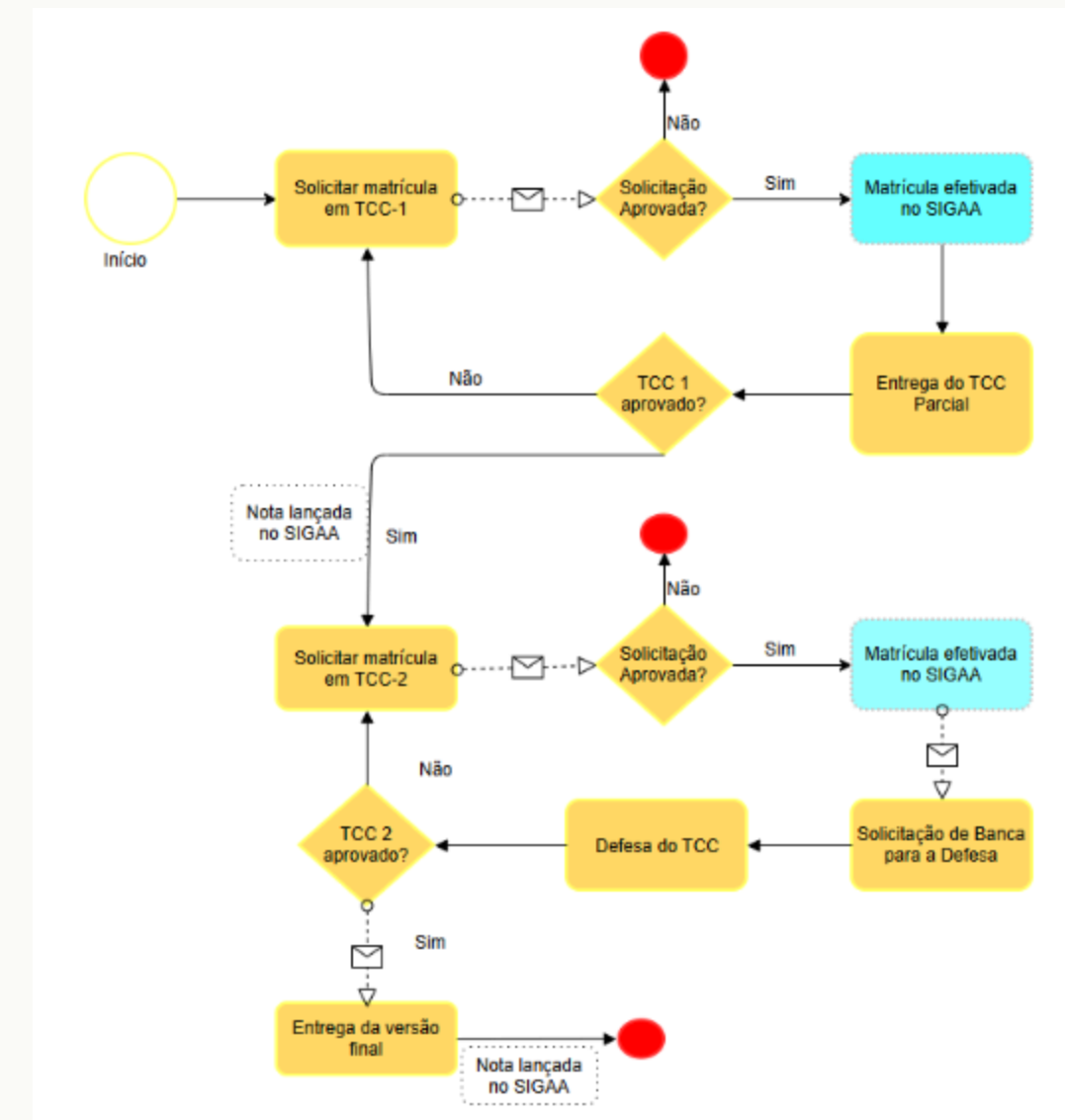
GERENCIAL

Relatórios de panorama ao coordenador, ao NDE e ao colegiado: defesas no prazo, distribuição por área, trabalhos com potencial de publicação.

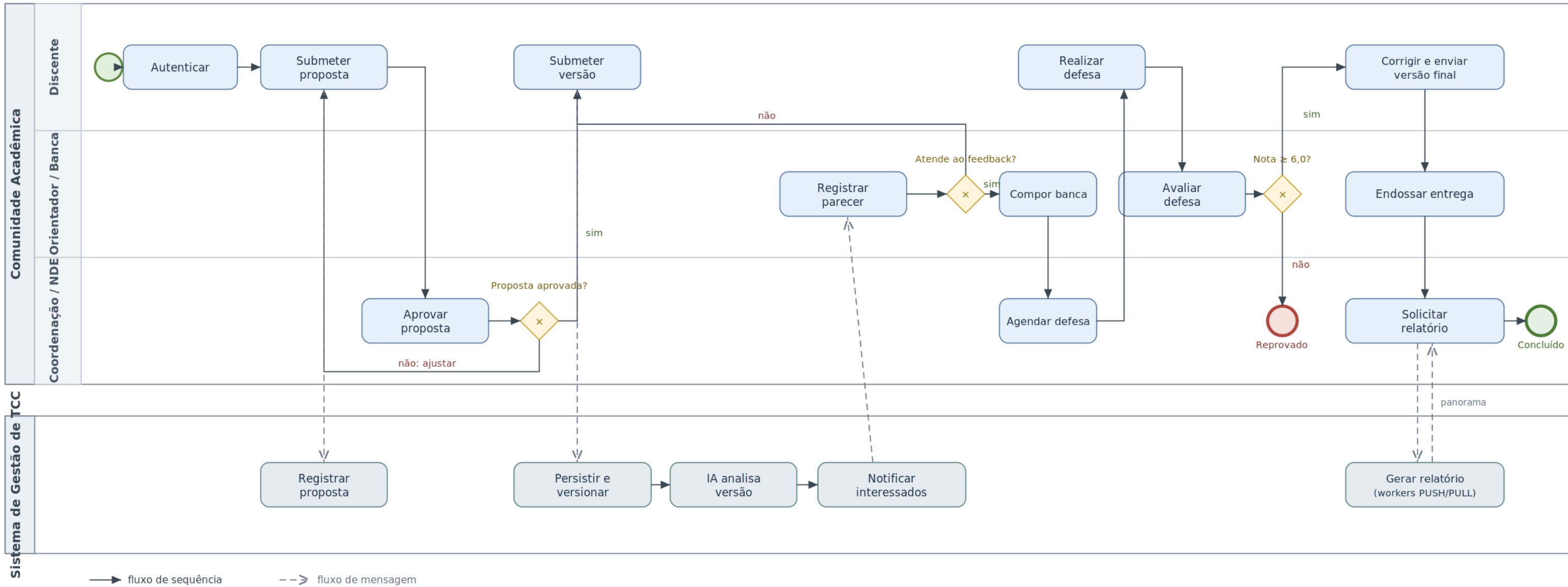
COMUNICAÇÃO

Descentralizada e sem padrão, um mesmo avaliador analisa vários alunos em formatos e meios diferentes, o que impede o acompanhamento sistemático.

Sem uma ferramenta integrada, o processo fica lento, complexo e custoso para todos.



Este é o processo que o sistema passa a orquestrar, de ponta a ponta.



Os agentes do TCC são autônomos, concorrentes e assíncronos: é um sistema distribuído.

01 • PROCESSOS INDEPENDENTES

Discente, orientador, banca e coordenação não compartilham memória.

02 • CONCORRÊNCIA

Atuam em paralelo, várias orientações e avaliações ao mesmo tempo.

03 • ASSINCRONIA

Raramente estão ativos no mesmo instante.

04 • MESSAGE PASSING

A única forma de cooperar é a troca explícita de mensagens.

Logo, desacoplamento, concorrência e assincronia são requisitos do problema, não opções.

O ZeroMQ é uma biblioteca de mensagens embutida no processo, sem broker central.

BIBLIOTECA, NÃO SERVIDOR roda dentro da própria aplicação.

BROKERLESS	fila por conexão em cada socket; os pares trocam direto, sem intermediário.
SMART SOCKETS	reconexão automática, enquadramento atômico e ordem de entrega.
DESEMPENHO	batching e zero-copy; pode superar sockets TCP diretos.
0 PREÇO	entrega de melhor esforço, pode descartar; não há exactly-once.

BROKERLESS · MALHA DIRETA



os pares trocam mensagens diretamente

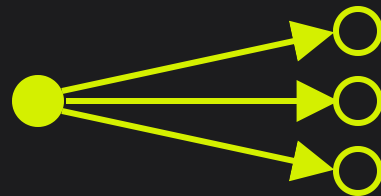
~~BROKER central~~

sem gargalo · sem ponto único de falha

Cada tipo de interação do sistema usa o padrão de socket adequado.

PUB / SUB

Um publica, muitos assinam;
filtragem por tópico.



A espinha dorsal dos eventos.

DEALER / ROUTER

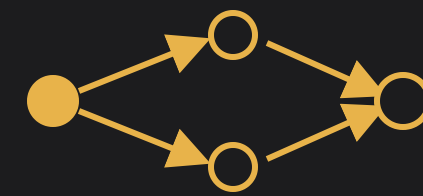
Request-reply assíncrono; endereça
pares específicos.



Usado no login.

PUSH / PULL

Pipeline que distribui tarefas a
workers.



Usado nos relatórios.

Nota: REQ/REP é a forma básica (síncrona) de request-reply; adotamos a variante assíncrona DEALER/ROUTER.

Escolhemos brokerless, coreografia e SAGA, cientes dos trade-offs.

TRANSPORTE

Brokerless (ZeroMQ)

em vez de broker (RabbitMQ / Kafka)

Direto entre pares; sem gargalo nem ponto único de falha.

COORDENAÇÃO

Coreografia

em vez de orquestração

Cada serviço reage aos eventos que assina; o comportamento global emerge, sem maestro.

TRANSAÇÕES

SAGA

em vez de 2PC (duas fases)

Transações locais com compensação; o 2PC bloquearia recursos num processo longo como o TCC.

O trade-off do brokerless (entrega fraca, sem persistência) é assumido e mitigado: idempotência por identificador + estado durável no MySQL.

Não partimos do zero: o que já existe e onde nos diferenciamos.

DORNELES (2023)

Gestão de TCC com REST + notificações via RabbitMQ. Mesmo domínio, nosso diferencial é ser brokerless e coreografado.

JEMS (SBC)

Submissão e avaliação de trabalhos. Dele vêm submissão de versões, revisores, pareceres padronizados e ciclo de resposta.

CERN (RDA3)

Validou o ZeroMQ em larga escala, substituindo o CORBA; avaliado melhor que Ice e Thrift. Respaldo da escolha.

MAJORDOMO (MDP)

Padrão ZeroMQ de entrega confiável (broker de serviço + heartbeat). O caminho da confiabilidade, deixado como evolução.

Frontend, uma borda de tradução e uma malha de serviços sobre o ZeroMQ.

FRONTEND

React/Vite: o que o usuário vê; fala HTTP.

BORDA BFF (PYTHON)

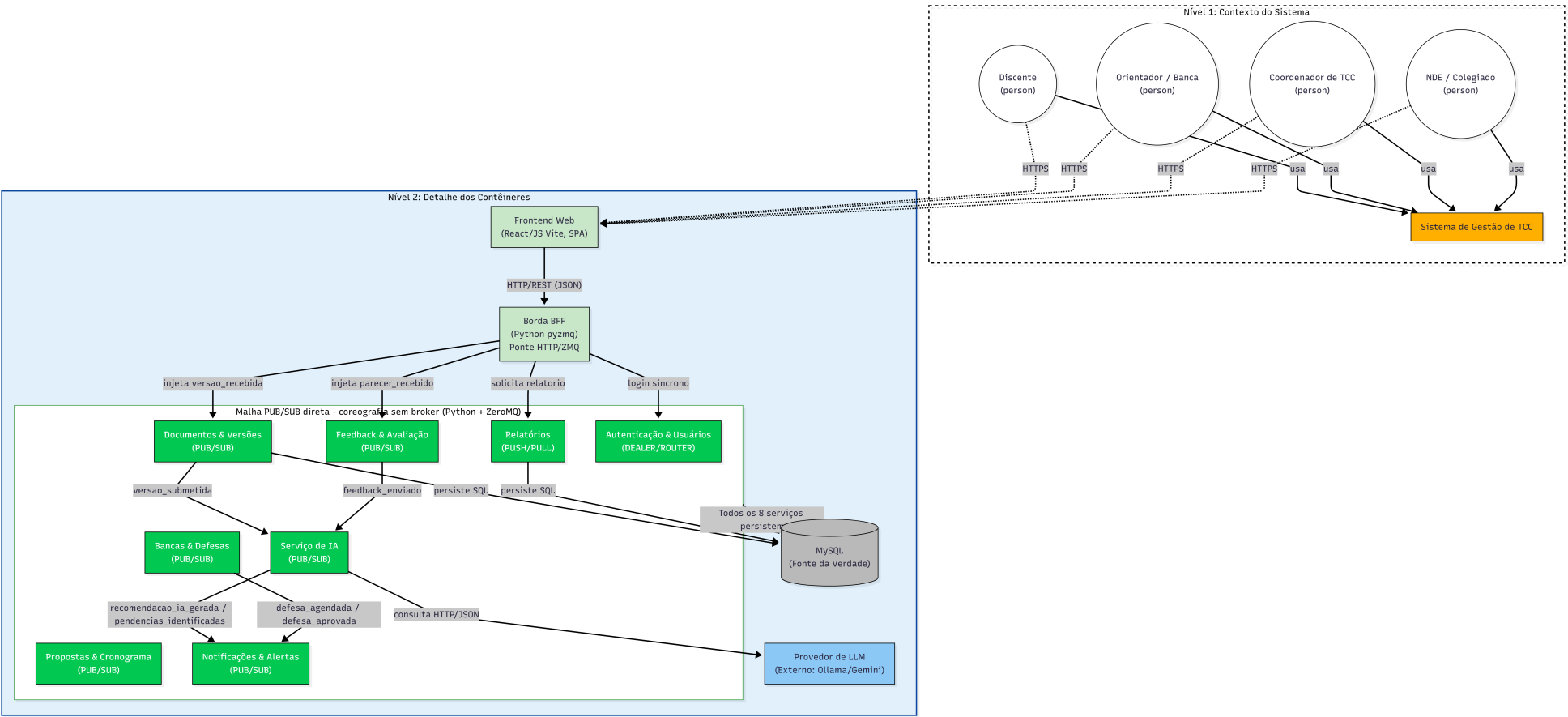
Traduz cada chamada HTTP em evento e devolve o resultado; não participa da coreografia.

MALHA DE 8 SERVIÇOS

Peers em Python/PyZMQ: o núcleo distribuído.

DADOS

MySQL: fonte da verdade. Provedor de LLM: Google Gemini (plugável).



**Cada domínio do processo virou um serviço independente,
com seu padrão.**

O diagrama apresenta a arquitetura de uma plataforma de negociação de criptomoedas, organizada em uma grade de 2x4 módulos. Cada módulo é representado por um retângulo cinza escuro com cantos arredondados, contendo um título principal em branco e uma descrição de funcionalidade em amarelo.

Módulo	Descrição
Autenticação & Usuários	DEALER/ROUTER
Propostas & Cronograma	PUB/SUB
Documentos & Versões	PUB/SUB
Feedback & Avaliação	PUB/SUB
Bancas & Defesas	PUB/SUB
Notificações & Alertas	PUB/SUB
Serviço de IA	PUB/SUB
Relatórios	PUSH/PULL

Toda mensagem é uma tupla, e o tempo é tratado com cuidado.

$$m = (e, o, i, t, d)$$

evento · operação · identificador · timestamp · dados (JSON)

i → **idempotência**

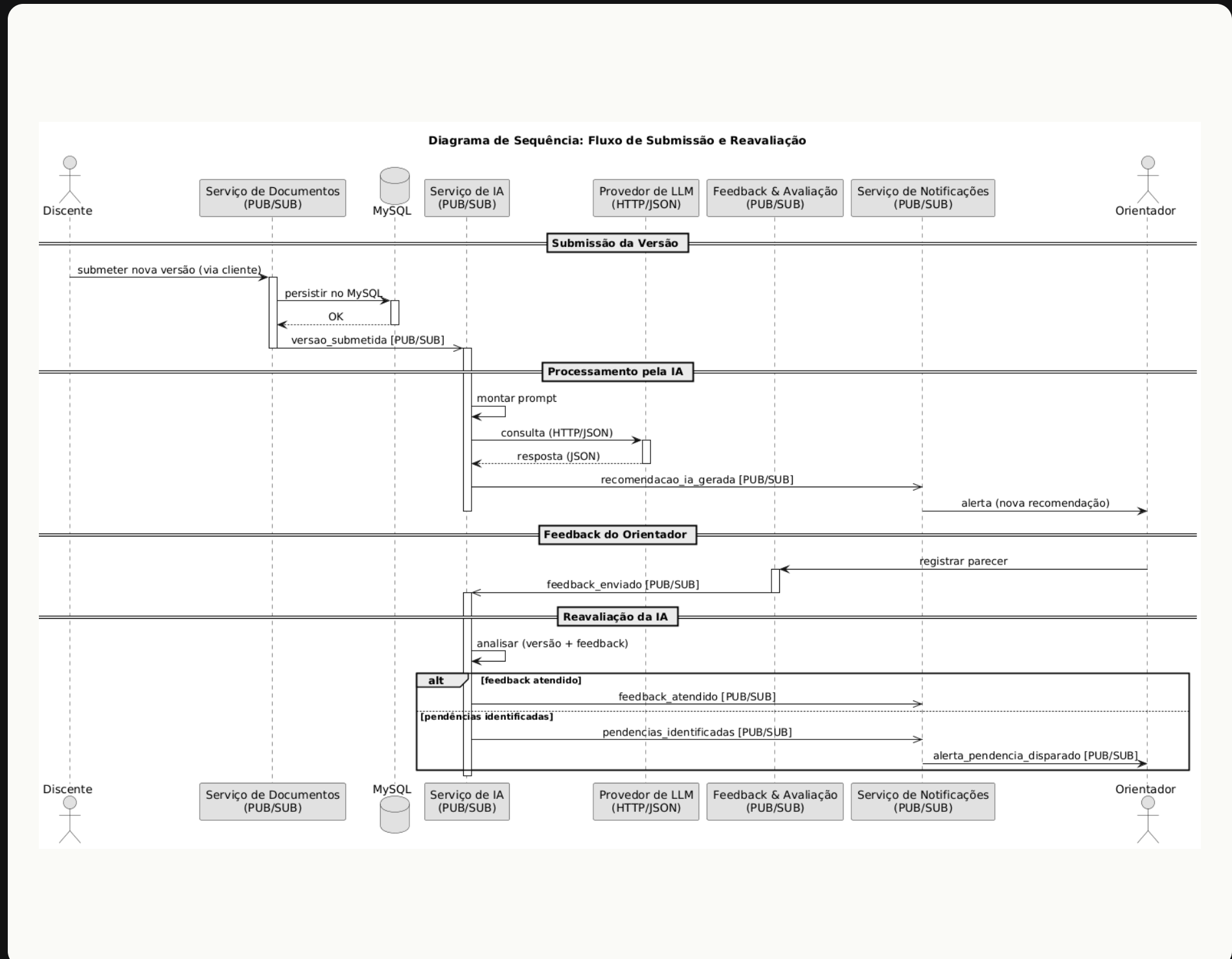
Reprocessar o mesmo evento não gera efeito duplicado.

t → **ordenação**

Hoje é um relógio físico. Ordenação independente dos relógios exigiria relógios lógicos, Lamport (acontece-antes) ou vetoriais (causalidade). Fica como evolução.

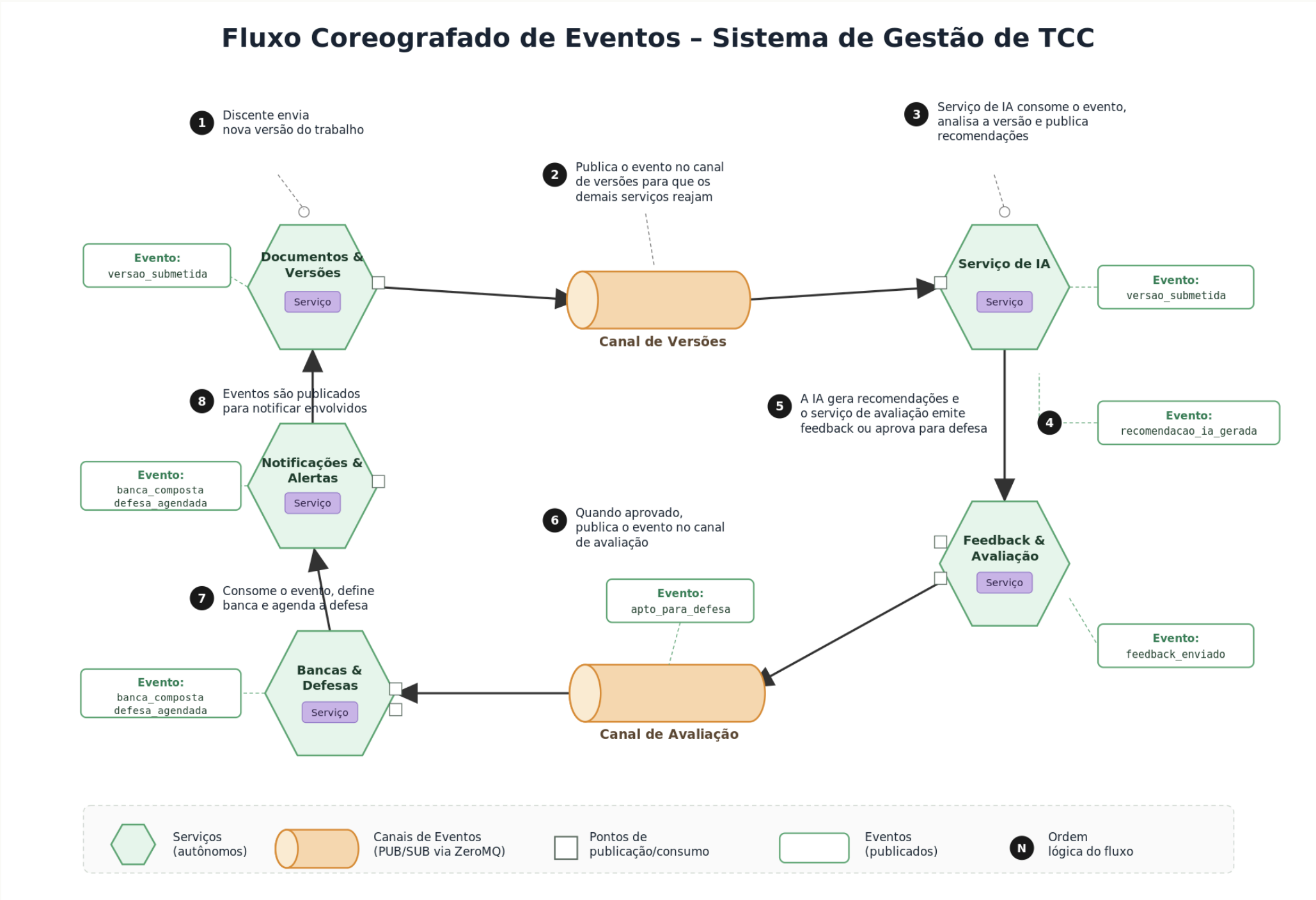
Um comando dispara uma reação em cadeia, sem coordenador central.

- 1 A borda injeta **versao_recebida**
- 2 Documentos persiste e publica **versao_submetida**
- 3 IA consulta o Gemini e publica **recomendacao_ia_gerada**
- 4 Feedback & Avaliação publica **feedback_enviado**; a IA reavalia (feedback_atendido / pendencias_identificadas)
- 5 Notificações publica **alerta_pendencia_disparado**



Ninguém coordena no centro.

O sistema inteiro em um mapa: serviços, canais e a ordem dos eventos.



O MySQL recupera a durabilidade que o transporte brokerless não oferece.

FORTE DA VERDADE

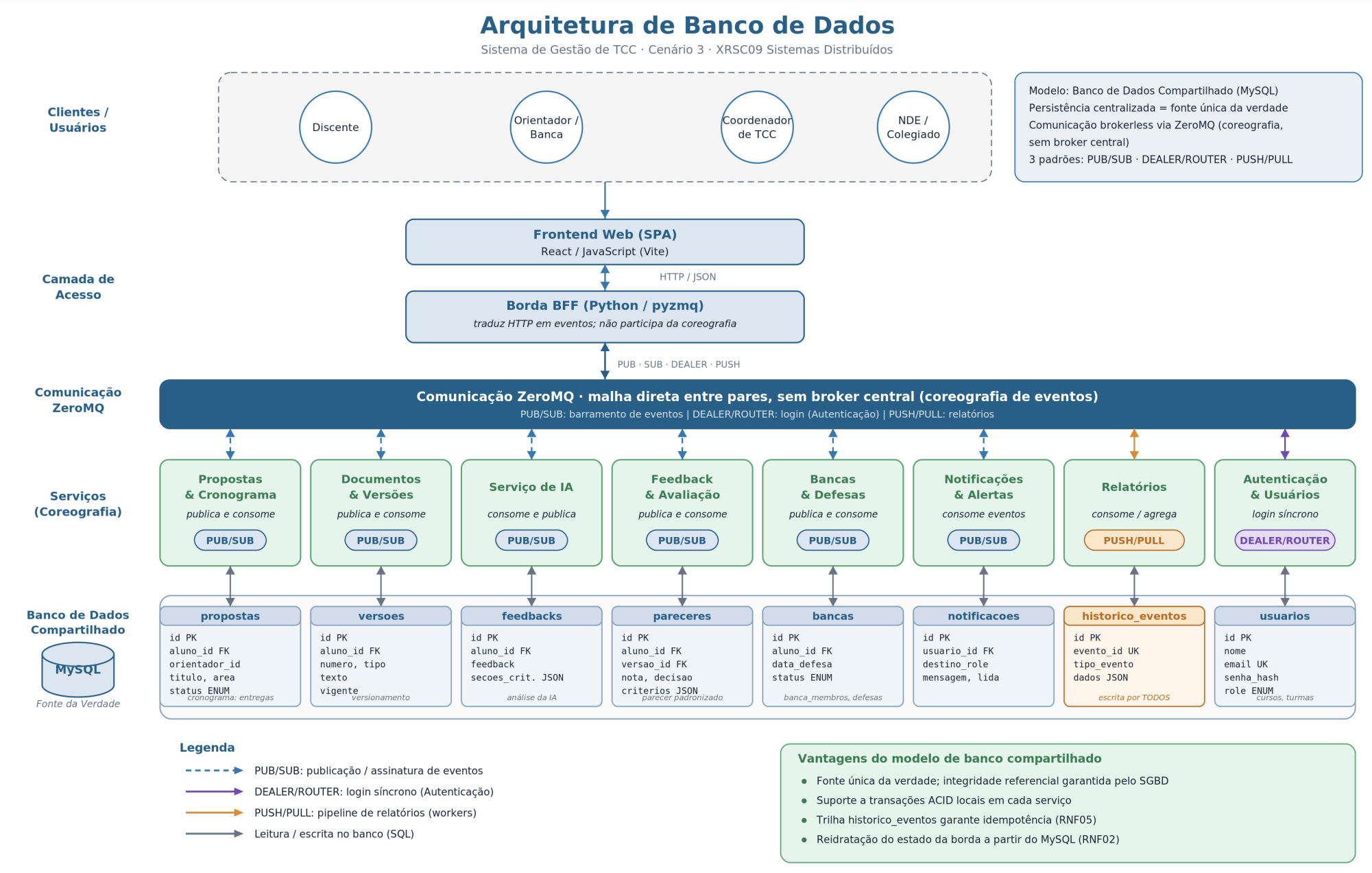
O MySQL é o estado compartilhado e consultável do sistema.

TRILHA DE EVENTOS

historico_eventos garante idempotência, alimenta os relatórios e serve de evidência.

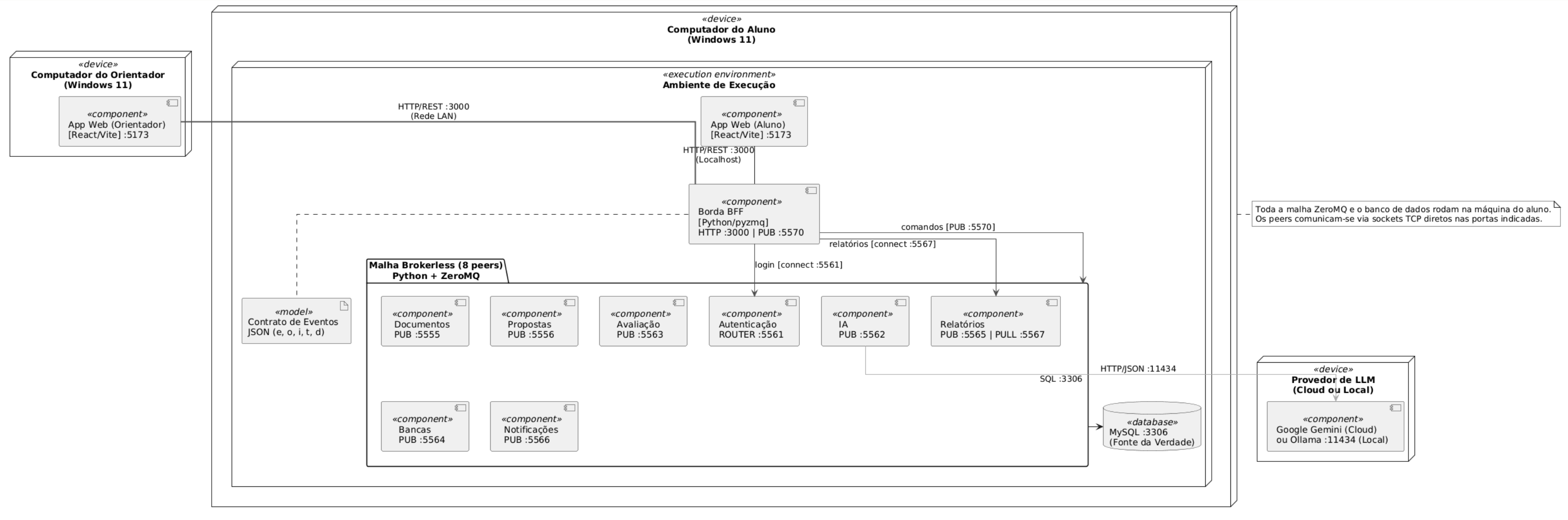
REIDRATAÇÃO

Um serviço que reinicia recompõe seu estado a partir do banco.



Os processos rodam em nós separados, ligados por rede.

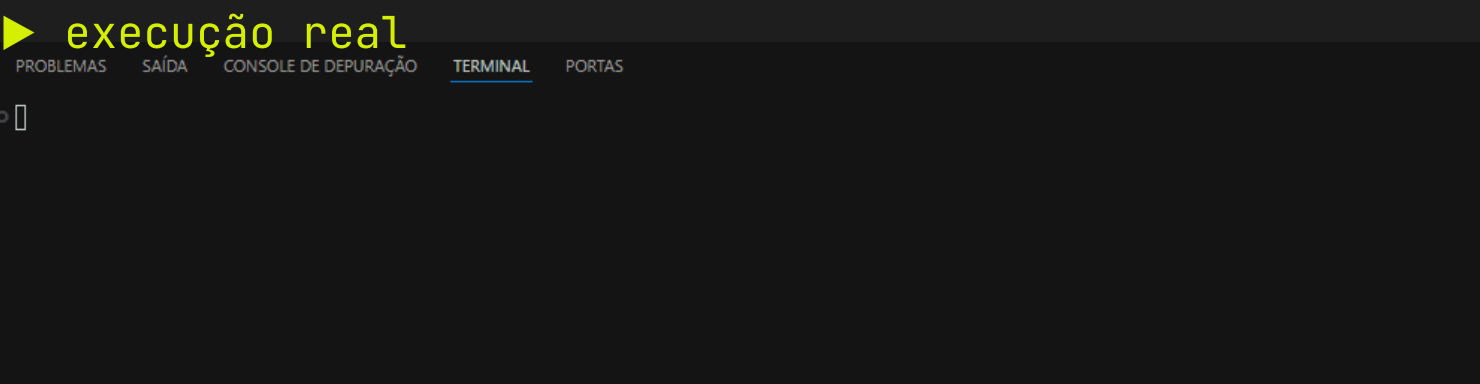
ZeroMQ por TCP entre os pares; HTTP da IA para o Gemini. No teste, duas máquinas, aluno e orientador.



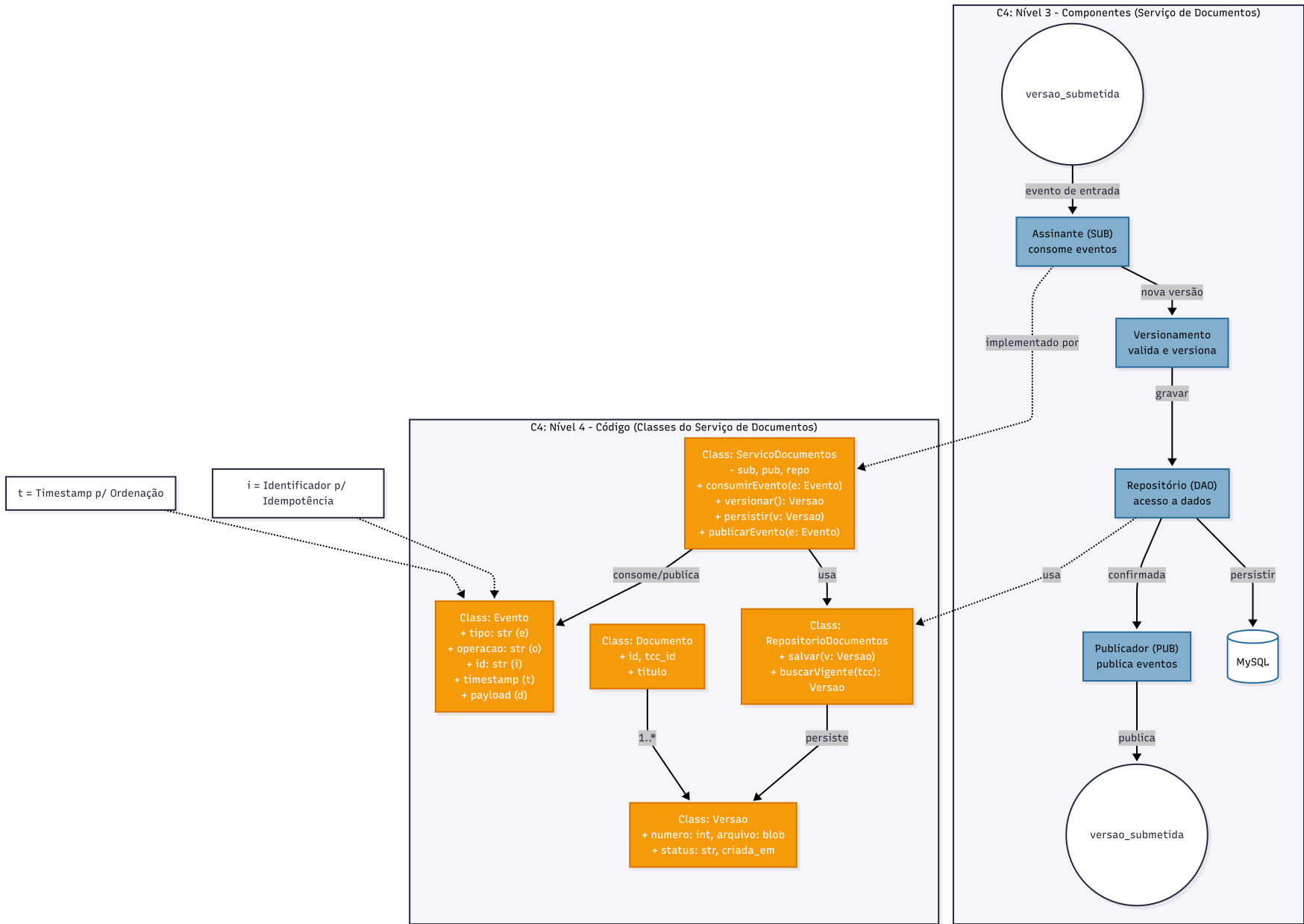
Cada peer segue o mesmo padrão: assina o evento, trata sem repetir e publica o próximo.

```
sub.connect(addr("gateway"))
sub.subscribe("versao_recebida")
pub.bind(addr("documentos"))

while True:
    dados = json.loads(sub.recv())
    if not repo.evento_novo(dados): # idempotência
        continue
    n = repo.salvar-versao(dados) # persiste
    pub.send(Evento(VERSAO_SUBMETIDA)) # publica
```

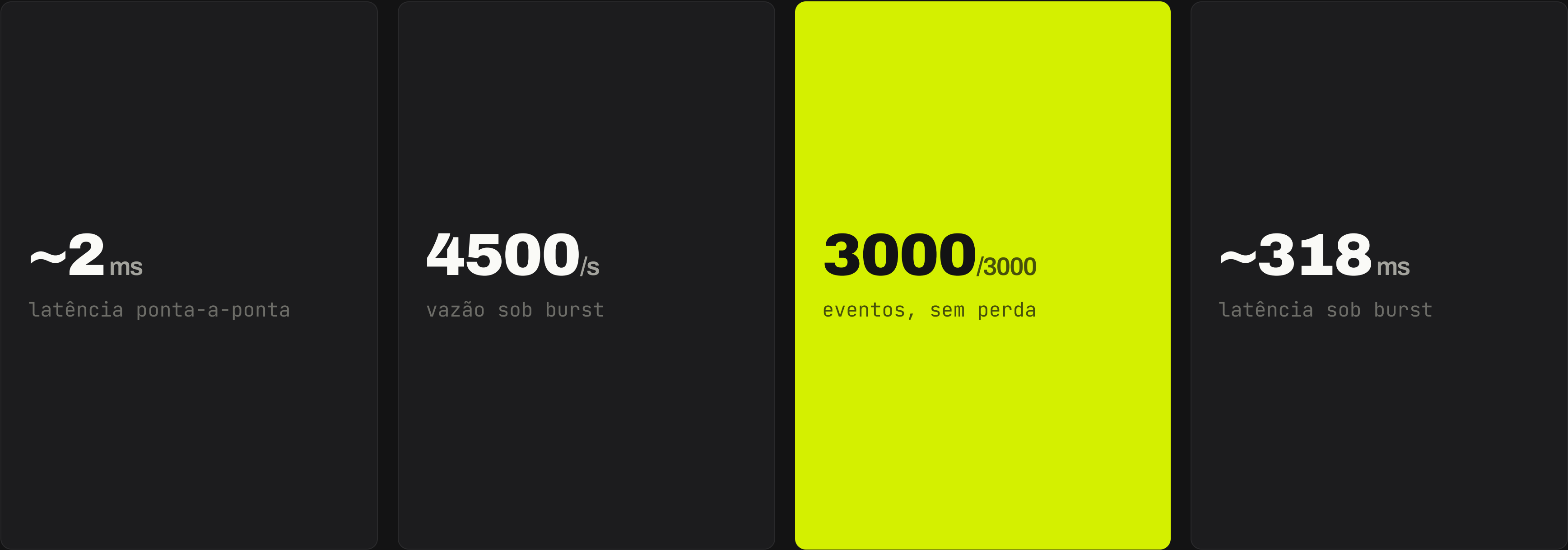


run.py sobe toda a malha + a borda num comando (offline por padrão) • o Gemini é a única chamada não-ZeroMQ (HTTP, com fallback simulado).



Para medir só a nossa comunicação, isolamos a IA com um provedor simulado.

Para medir o transporte (ZeroMQ) sem a latência de rede e de inferência do Gemini contaminar o número. O Gemini roda de verdade na demonstração.



Confirma os RNFs: comunicação assíncrona, concorrência, escalabilidade e idempotência • medido em 1 máquina, ZeroMQ em loopback.

Cada dor do início virou um serviço, um evento e um dado durável.

O PROBLEMA

- Comunicação dispersa
- Versões soltas por e-mail
- Revisões sem padrão
- Sem visão gerencial
- Esforço com correções básicas

A SOLUÇÃO

- Malha de eventos coreografada
- Versionamento no serviço de Documentos
- Pareceres padronizados (Feedback & Avaliação)
- Serviço de Relatórios (PUSH/PULL)
- Apoio da IA (Google Gemini)

DEMONSTRAÇÃO AO VIVO

Cenário de submissão e revisão, a cadeia de eventos disparando sozinha, a recomendação do Gemini e a notificação.